

Lecture No (7)

Web Applications Improvement

1. Introduction

- One of the success-critical quality characteristics of Web applications is system performance.
- What users primarily perceive is the time that elapses between sending a request and receiving an answer.
- For e-commerce Web sites it could result in losing potential customers

2. What Is Performance?

- *response time (R)*,
 - time between sending a request to the system and receiving an answer from that system
- *throughput (X)*,
 - how many requests can be served per time unit
- *Utilization (U)*
 - percentage in which the system was busy.

3. What Characterizes the Performance of Web Applications?

- traditional" desktop applications:
 - time remaining (e.g., progress bar)
- Unfortunately, this transparency is not available for most Web applications
 - impact of poor performance is painfully perceived
- Users cannot perceive an obvious difference
 - static HTTP page, and clicking a link that triggers the
 - dynamic creation of a page,
 - but the response times can be very different
- No way to guarantee the quality of a service
 - they can't guarantee sufficient bandwidth all the way to the client
- It is, therefore, important not only to test Web applications on different platforms, but also to run performance tests over several network connections and client types

4. System Definition and Indicators

- When analyzing Web applications:
 - Web applications themselves,
 - the Web servers and application servers (hardware and software) to be used,
 - the network connections between servers,
 - network connections between clients and servers,
 - and the clients themselves

5. Characterizing the Workload

- Workload model, we have to consider two dimensions:
 1. executability of the workload
 2. restriction to actual workload or artificial workload
- Synthetic workload
 - requests identical to those occurring in the real world
- single-class workload model
- multi-class workload model
- non-executable workload models

6. Analytical Techniques

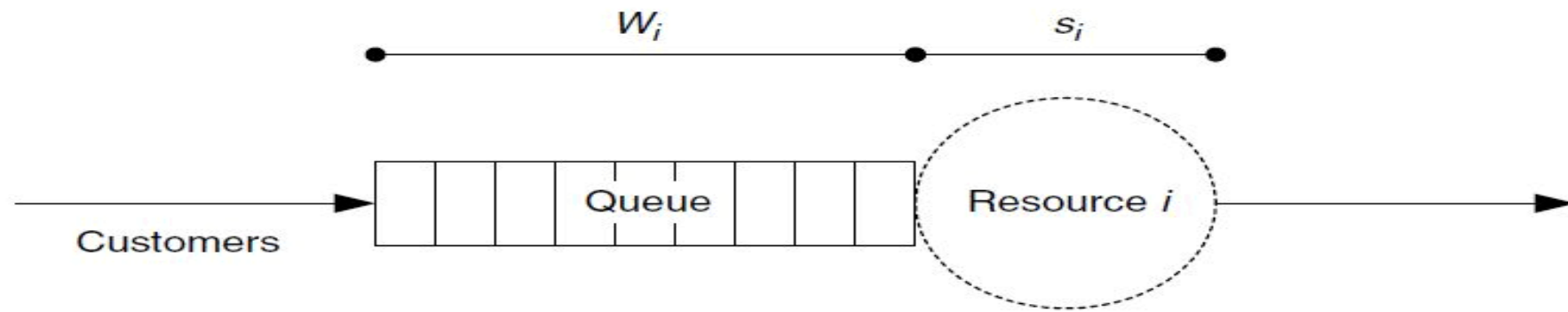
- analytical techniques are well suited for an initial estimate of a Web application's performance in the early development phases
- Operational Analysis
- Queuing Networks and Simulation Models
- Measuring Approaches

6.1 Operational Analysis

- An operational analysis takes some basic measurable (operational) quantities:
 - primarily the number of completed requests (C)
 - Defined observation period (T)
- throughput of station k is $X = C_k / T$
 - Example: Let's look at a Web server consisting of one CPU and two disks. The system has been observed by measurements for an hour. A total of 7200 requests has been completed during this period
 - $X = C/T = 7200/3600$ requests per second = 2 requests per second
- Little's Law (Little 1961) defines a relationship between
 - response time,
 - throughput, and the
 - number of requests in the system (N)
- If a request spends R time units on average in the system, and if X requests per time unit are completed on average, then an average of $N = R * X$ requests must be spending time in the system.
 - Example: Let's assume that the mean response time of all 7200 requests found to be 0.9 seconds. $N = R * X = 0.9 * 2 = 1.8$ requests spent time in the Web server during the observation period.

6.2 Queuing Networks and Simulation Models

- modeling technique for all systems characterized by a set of components that send requests (called sources), and a set of components that process requests
- waiting time - W_i and with service time - s_i

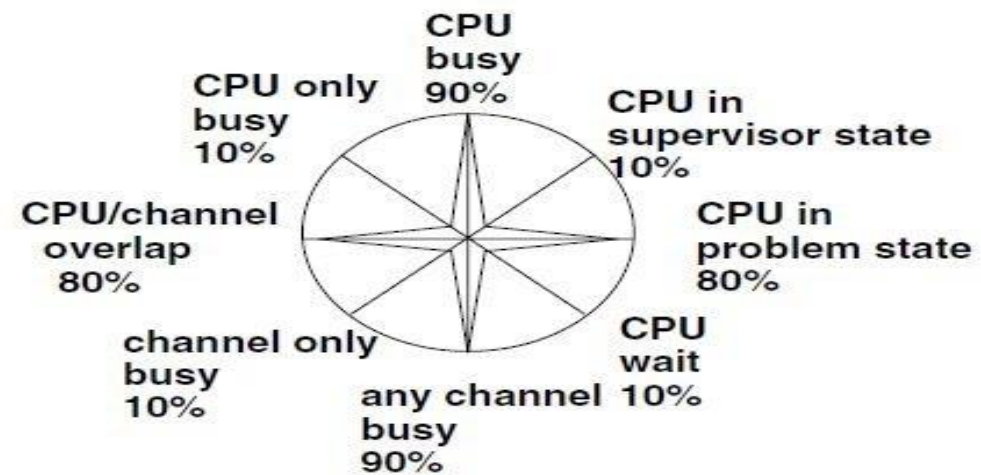
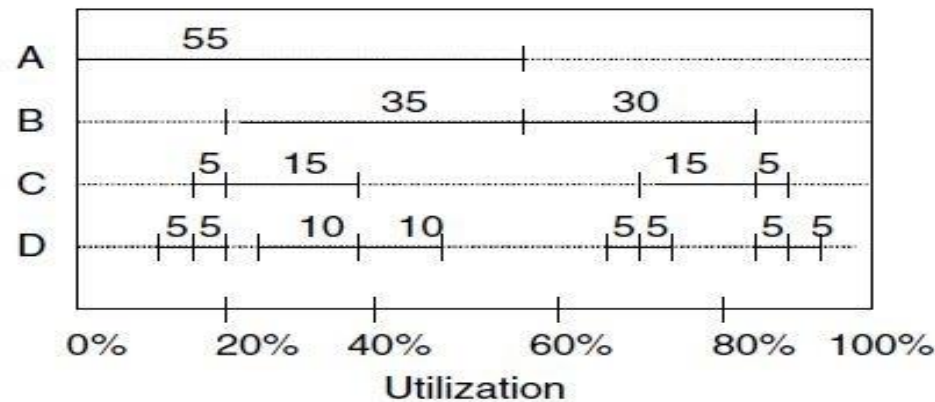


6.3 Measuring Approaches

- use timers to measure the time at welldefined system points
- use counters to count the occurrence of certain events
- Using measuring software, hardware
- Load generators
 - generate synthetic or artificial workload, allowing the analyst to study the system behavior
- *benchmark*
 - *is an artificial* workload model that serves to compare systems
 - One example is the SpecWeb benchmark (see <http://www.spec.org>)

7 Representing and Interpreting Results

- The last step the representation and interpretation of the results.



8 Performance Optimization Methods

- targeted at shortening response times
- these goals can be achieved only provided we identify and remove the bottleneck in a system
- bottleneck can basically be removed by
 - reducing the load on the bottleneck component
 - (e.g., reducing the transmitted file sizes to remove load from the network)
 - increasing the performance of the component itself
 - (e.g., increasing the bandwidth of the network connection)
- Once we have removed the primary bottleneck, we typically observe that another component will become a bottleneck;
- Optimal performance system is a balanced system whose all components are (approximately) requires equal performance

8.1 Acceleration Within a Web Application

- This class includes by definition all methods that aim at shortening the execution time of the application on the server

Load Balancing Between Client and Server

- we have to decide how much of the application logic should be processed at the client and how much at the server.
- For example, checking data input in forms could be done at the client. These controls are relevant both for security reasons and performance reasons.

Embedding Server-side Applications

- Web servers offer a CGI (Common Gateway Interface) that can be used to forward HTTP requests to other programs in the server for further processing. From the performance perspective, we could investigate alternative embedding possibilities, e.g., by using servlets (see also Chapter 6).

Pre-generating Web Content

- The dynamic generation of pages based on information from other applications (typically databases) is a:
 - computing-intensive
 - time-consuming process.
- Thus pre-generate popular pages and then make them available as static pages

Adapting an HTTP Response to the Client's Capabilities

- when embedding multimedia contents, we should take the capabilities of clients into account to ensure, for instance, that both the server and the network won't get overloaded by transmitting large data sets, which wouldn't be displayed correctly in the client anyway

8.2 Reducing Transmission Time

- Web Caching and Content Pre-fetching
- Web Server Replication and Load Balancing

Web Caching and Content Pre-fetching

- Web browsers store frequently requested pages on the hard disk
- When the client visits page again, it fetches from its cache
- continuation of this concept are Web proxy servers
 - they serve replies to client requests from their cache rather than from the server.
- Web content pre-fetching
 - based on the same idea, but it additionally tries to predict the pages a user will access next
- A well known and commonly used caching tool is for example SQUID (<http://www.squid-cache.org>)
- Drawback
 - A poorly configured proxy or cache server can even cause response times to increase.

Web Server Replication and Load Balancing

- redirects it to a replicate of that server
- so-called mirror sites - alternative servers
- drawback is that users normally don't know what server offers the best performance
- implicit mirroring lets users access one URL of a server, request is internally forwarded to a replicate server
- Surrogate servers
 - reply to the request from its cache, and accesses the original Web server in the event of a cache miss

8.3 Server Tuning

Is aimed at improving the execution platform of a Web application

1. Hardware upgrades

- main memory expansion,
- CPU acceleration,
- Faster disks

2. Optimized settings in the Web server:

- <http://httpd.apache.org/docs/misc/perf.html>
- <http://www.microsoft.com/serviceproviders/whitepapers/tuningiis.asp>

The End